

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №1
дисциплины
«Искусственный интеллект в профессиональной сфере»
Вариант 1

Выполнил:
Бабенко Артём Тимофеевич
3 курс, группа ИВТ-б-о-22-1,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Проверил:
Ассистент департамента цифровых,
робототехнических систем и
электроники Богданов С.С

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2024 г.

Тема: исследование методов поиска в пространстве состояний

Цель: приобретение навыков по работе с методами поиска в пространстве состояний с помощью языка программирования Python версии 3.x

Порядок выполнения работы:

Задание 1. Воспользуйтесь сервисом Google Maps или аналогичными картографическими приложениями, чтобы выбрать на карте более 20 населённых пунктов, связанных между собой дорогами. Постройте граф, где узлы будут представлять населённые пункты, а рёбра — дороги, соединяющие их. Вес каждого ребра соответствует расстоянию между этими пунктами. Выберите начальный и конечный пункты на графе. Определите минимальный маршрут между ними, который должен проходить через три промежуточных населённых пункта. Покажите данный путь на построенном графе.

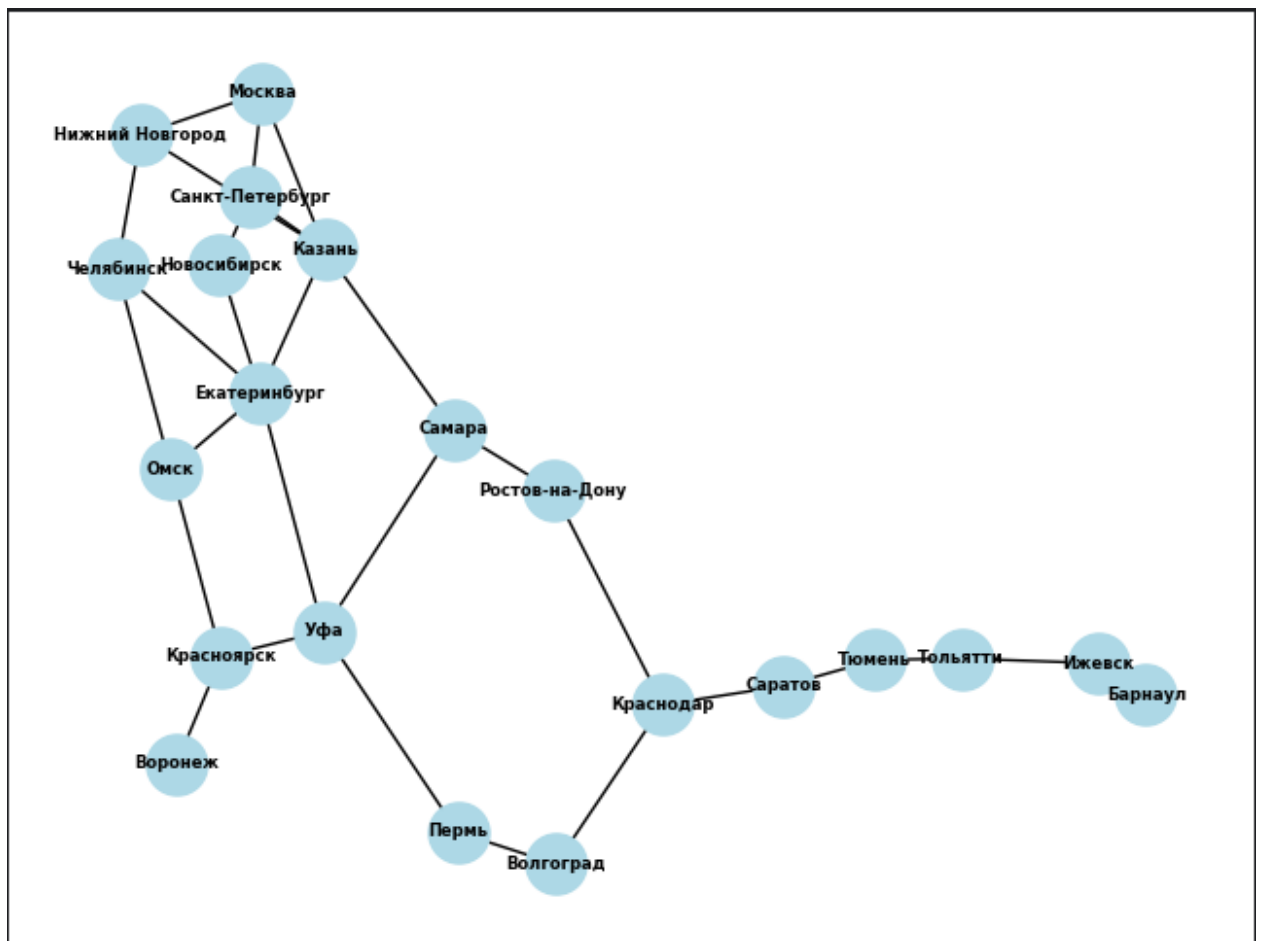


Рисунок 1 – Построенный граф

Начальная точка маршрута – Москва

Пункт назначения – Красноярск

С помощью программы, написанной на языке Python был найден кратчайший маршрут между этими двумя городами, проходящий как минимум через три других города.

Результат работы программы:

```
D:\Gitlab\AI\AI-in-prof\.venv\Scripts\python.exe D:\Gitlab\AI\AI-in-prof\.vscode\1.py
Минимальный маршрут: ['Москва', 'Казань', 'Екатеринбург', 'Уфа', 'Красноярск']
Общая длина маршрута: 4500 км
```

Рисунок 2 – Результат работы программы

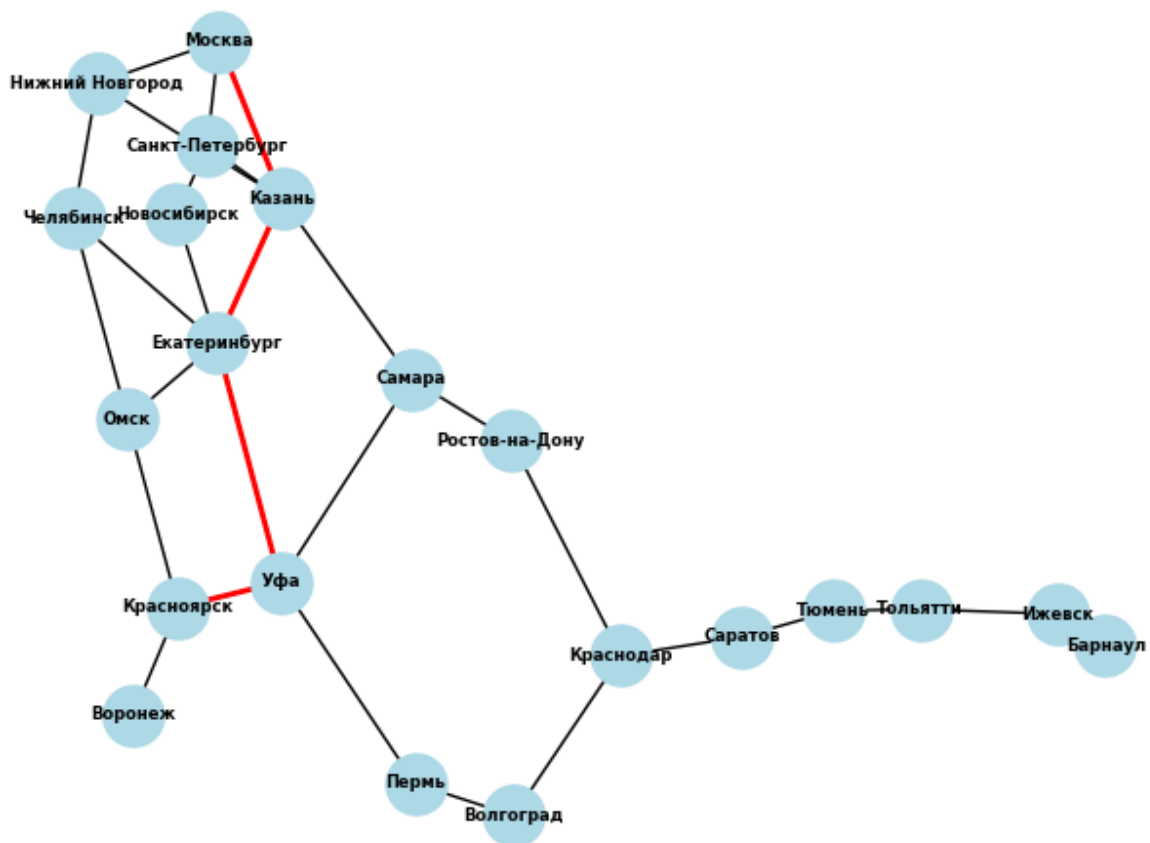


Рисунок 3 – Маршрут на графе

Задание 2. Методом полного перебора решите задачу коммивояжёра.

С помощью программы, написанной на языке Python найден маршрут из начальной точки в неё же, проходящий через заданные точки

Результат работы программы:

```
D:\Gitlab\AI\in-prof\.venv\Scripts\python.exe D:\Gitlab\AI\in-prof\.vscod\2.py
Оптимальный маршрут: ['Москва', 'Санкт-Петербург', 'Казань', 'Самара', 'Ростов-на-Дону', 'Новосибирск', 'Екатеринбург', 'Омск', 'Челябинск', 'Нижний Новгород', 'Москва']
Общая длина маршрута: 18700 км

Process finished with exit code 0
```

Рисунок 4 – Результат работы программы

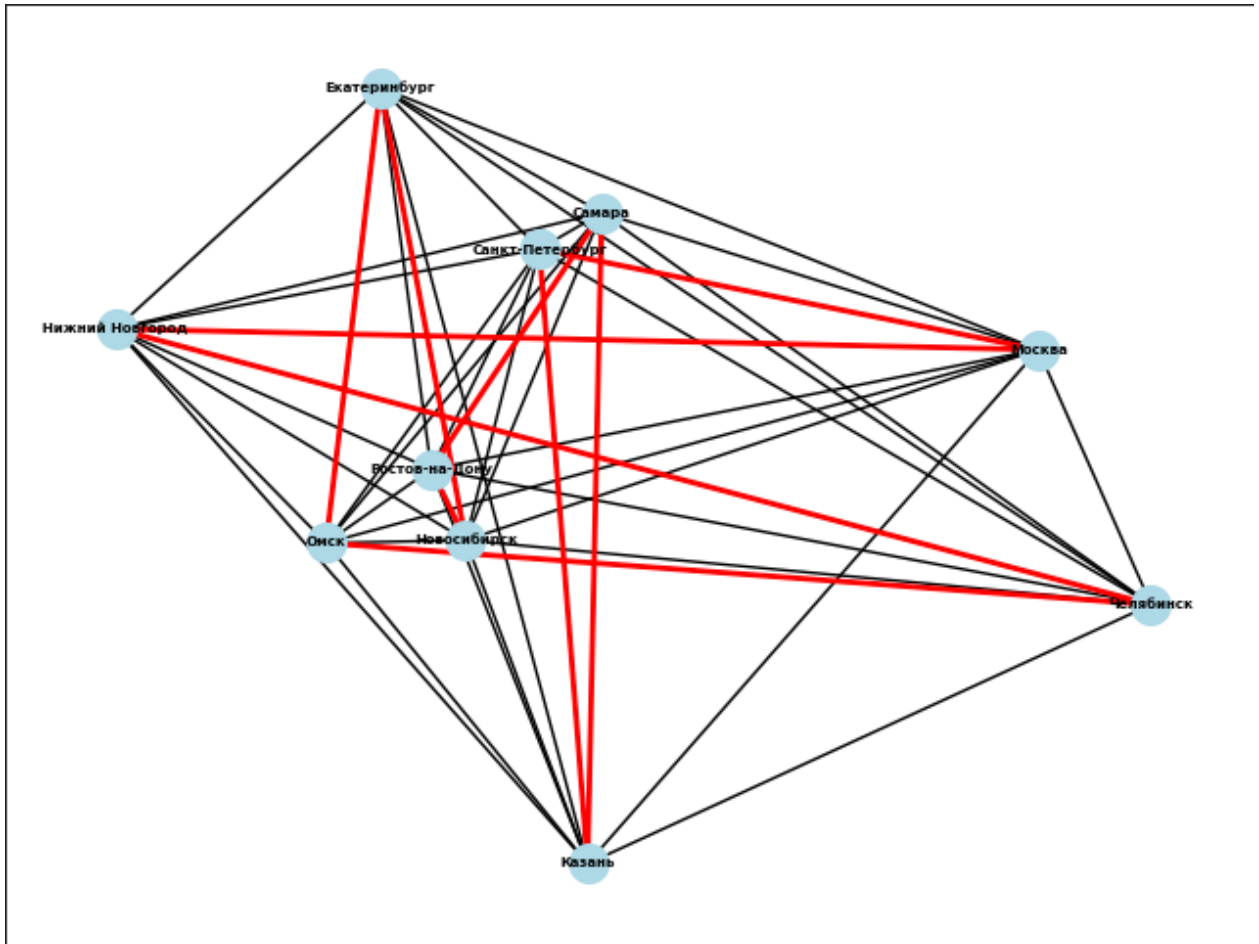


Рисунок 5 – Маршрут на графе. Число городов уменьшено, что бы
были возможны вычисления

Ответы на контрольные вопросы:

1. Что представляет собой метод "слепого поиска" в искусственном интеллекте?

Метод "слепого поиска" исследует пространство возможных решений методом проб и ошибок, не обладая информацией о том, насколько близко каждое принятое решение к финальной цели.

2. Как отличается эвристический поиск от слепого поиска?

Эвристический поиск, в отличие от слепого, использует дополнительные знания или "эвристики" для направления процесса поиска.

3. Какую роль играет эвристика в процессе поиска?

Эвристика позволяет «направлять» процесс поиска.

4. Приведите пример применения эвристического поиска в реальной задаче.

Один из наиболее известных примеров такого поиска - алгоритм A^* , который находит наиболее оптимальный путь к цели, основываясь на заранее заданных критериях и предположениях.

5. Почему полное исследование всех возможных ходов в шахматах затруднительно для ИИ?

При решении данной задачи встает вопрос о ресурсах - времени и памяти. Для многих задач эти ограничения могут быть преодолены, но в контексте шахмат они становятся критическими факторами.

6. Какие факторы ограничивают создание идеального шахматного ИИ?

Необходимость быстро обрабатывать информацию и принимать решения в условиях когда время может быть ограничено делает задачу создания эффективного шахматного ИИ особенно сложной.

7. В чем заключается основная задача искусственного интеллекта при выборе ходов в шахматах?

Центральная задача в создании шахматного ИИ - это выбор оптимальных ходов.

8. Как алгоритмы ИИ балансируют между скоростью вычислений и нахождением оптимальных решений?

Некоторые алгоритмы могут быстро находить решения, но они могут быть далеки от оптимальных. Другие, напротив, способны находить наилучшие решения, но требуют значительных временных и ресурсных затрат.

9. Каковы основные элементы задачи поиска маршрута по карте?

При решении задачи поиска маршрута по карте требуется найти оптимальное решение, при котором расстояние будет минимальное.

10. Как можно оценить оптимальность решения задачи маршрутизации на карте Румынии?

Для оценки оптимальности можно использовать длину пути, которая при оптимальном решении должна быть минимальной.

11. Что представляет собой исходное состояние дерева поиска в задаче маршрутизации по карте Румынии?

Исходное состояние дерева представляет из себя корень дерева – город Арад.

12. Какие узлы называются листовыми в контексте алгоритма поиска по дереву?

Листовыми называются узлы, имеющие родительский узел.

13. Что происходит на этапе расширения узла в дереве поиска?

В дерево добавляются дочерние вершины узла, который расширяется на текущем шаге.

14. Какие города можно посетить, совершив одно действие из Арада в примере задачи поиска по карте?

Можно посетить следующие города: Сибиру, Тимишоара, Зеринд.

15. Как определяется целевое состояние в алгоритме поиска по дереву?

Целевое состояние задается как условие задачи.

16. Какие основные шаги выполняет алгоритм поиска по дереву?

Алгоритм поиска по дереву раскрывает узлы, добавляя в дерево новые и сравнивает текущее состояние с целевым.

17. Чем различаются состояния и узлы в дереве поиска?

Состояние (State) - это представление конфигурации в нашем пространстве поиска, например, города на карте Румынии, где мы находимся, или конфигурация плиток в головоломке.

Узел (Node) - это структура данных о состоянии, содержащая само состояние, а также другие данные: указатель на родительский узел, действие, стоимость пути и глубину узла в дереве

18. Что такое функция преемника и как она используется в алгоритме поиска?

Функция генерирует преемников уже изученных состояний.

19. Какое влияние на поиск оказывают такие параметры, как b (разветвление), d (глубина решения) и m (максимальная глубина)?

1) b (максимальный коэффициент разветвления) - показывает, сколько дочерних узлов может иметь один узел.

2) d (глубина наименее дорогого решения) - определяет, насколько далеко нужно спуститься по дереву для нахождения оптимального решения.

3) m (максимальная глубина дерева) - показывает, насколько глубоко можно в принципе спуститься по дереву. В некоторых случаях это значение может быть бесконечным.

20. Как алгоритмы поиска по дереву оцениваются по критериям полноты, временной и пространственной сложности, а также оптимальности?

Полнота означает, что алгоритм находит решение, если оно существует. Отсутствие решения возможно только в случае, когда задача невыполнима.

Временная сложность измеряется количеством сгенерированных узлов, а не временем в секундах или циклах ЦПУ. Она пропорциональна общему количеству узлов.

Пространственная сложность относится к максимальному количеству узлов, которые нужно хранить в памяти в любой момент времени. В некоторых алгоритмах она совпадает с временной сложностью. Пространственная сложность зависит от того, какие узлы сохраняются в памяти. В некоторых алгоритмах необходимо сохранять все сгенерированные узлы, тогда как в других узлы могут быть отброшены после проверки, не

требуя длительного сохранения в памяти. Это различие позволяет временной и пространственной сложности отличаться друг от друга.

Оптимальность - это способность алгоритма всегда находить наилучшее решение с минимальной стоимостью, если таковое существует. Она не связана напрямую с временной или пространственной сложностью, которые измеряют количество узлов. Оптимальность не гарантирует быстрое действие или экономию памяти, а лишь обеспечивает нахождение решения с наименьшей стоимостью

21. Какую роль выполняет класс Problem в приведенном коде?

Этот класс служит шаблоном для создания конкретных задач в различных предметных областях. Каждая конкретная задача будет наследовать этот класс и переопределять его методы

22. Какие методы необходимо переопределить при наследовании класса Problem ?

Необходимо переопределить следующие методы:

```
def actions(self, state): raise NotImplementedError
def result(self, state, action): raise NotImplementedError
def is_goal(self, state): return state == self.goal
def action_cost(self, s, a, s1): return 1
def h(self, node): return 0
```

23. Что делает метод is_goal в классе Problem ?

Метод is_goal проверяет, достигнуто ли целевое состояние.

24. Для чего используется метод action_cost в классе.

Метод action_cost предоставляет стандартную реализацию для стоимости действия и эвристической функции соответственно

25. Какую задачу выполняет класс Node в алгоритмах поиска?

Определение класса Node , который представляет узел в дереве поиска.

26. Какие параметры принимает конструктор класса Node ?

Принимает текущее состояние (`state`), ссылку на родительский узел (`parent`), действие, которое привело к этому узлу (стоимость пути (`action`), и `path_cost`).

27. Что представляет собой специальный узел `failure` ?

Специальный узел `failure` обозначает неудачу в поиске.

28. Для чего используется функция `expand` в коде?

Функция `expand` расширяет узел, генерируя дочерние узлы.

29. Какая последовательность действий генерируется с помощью функции `path_actions` ?

Функция `path_actions` возвращает последовательность действий, которые привели к данному узлу.

30. Чем отличается функция `path_states` от функции `path_actions` ?

Функция `path_states` возвращает последовательность состояний, ведущих к данному узлу, а функция `path_actions` возвращает последовательность действий, которые привели к данному узлу.

31. Какой тип данных используется для реализации `FIFOQueue`?

`FIFOQueue` - это очередь, где первый добавленный элемент будет первым извлеченным. Она реализуется с помощью `deque` из модуля `collections`. `deque` это обобщенная версия стека и очереди, которая поддерживает добавление и удаление элементов с обоих концов.

32. Чем отличается очередь `FIFOQueue` от `LIFOQueue` ?

`LIFOQueue` - это стек, где последний добавленный элемент будет первым извлеченным. В этом случае для реализации используется обычный список Python (`list`). Добавление и извлечение элементов происходит с одного конца списка.

33. Как работает метод `add` в классе `PriorityQueue` ?

`add` - метод для добавления элемента в очередь. Каждый элемент добавляется в кучу с его приоритетом, определенным функцией `key`.

34. В каких ситуациях применяются очереди с приоритетом?

Очереди с приоритетом используются в алгоритмах поиска кратчайшего пути, таких как A*.

35. Как функция `heappop` помогает в реализации очереди с приоритетом.

Метод для извлечения элемента с наименьшим значением $f(item)$ из очереди использует функцию `heappop` из модуля `heapq`, что гарантирует извлечение элемента с наименьшим приоритетом.

Вывод: в ходе выполнения работы приобретены навыки работы с методами поиска в пространстве состояний с помощью языка программирования Python версии 3.x